Klyvix

REST & SSE API Specification & Developer Guide

OPENAPI SPEC • V2.0

PROTOCOL	BASE URL	INTERACTIVE DOCS	REDOC SPEC
HTTP/2 • SSE Streaming	https://klyvix-177846015206.europe-west1.run.app	/docs (FastAPI Swagger)	/redoc (Formal Reference)

1 Authentication & Global Headers

Protected endpoints require a Bearer token in the `Authorization` header:

```
Authorization: Bearer <jwt_token>
Content-Type: application/json
Accept: text/event-stream, application/json
```

2 Core API Endpoints Reference

2.1 Streaming Chat Endpoint (`POST /api/chat/{persona_id}`)

Initiates a real-time Server-Sent Events (SSE) token stream for any of the 6 specialized personas (`study_mentor`, `code_reviewer`, `code_colleague`, `document_analyzer`, `resume_reviewer`, `research_assistant`).

```
// Request Payload: POST /api/chat/study_mentor
{
  "message": "Explain QuickSort algorithm step by step",
  "history": [
    { "role": "user", "content": "Hello" },
    { "role": "assistant", "content": "Welcome to Study Mentor. What shall we learn?" }
  ],
  "file_ids": []
}

// Response Stream: text/event-stream
data: {"token": "QuickSort"}
data: {"token": " is a divide-and-conquer"}
data: {"token": " algorithm..."}
data: [DONE]
```

2.2 PDF Document Upload & Indexing (`POST /api/upload`)

Uploads a PDF document, extracts text via `pypdf`, creates semantic vector embeddings, and stores vectors in Qdrant Cloud.

```
// Form-Data: POST /api/upload
file: <binary_pdf_data>

// Response (200 OK):
{
  "file_id": "doc_9f83a7c2-19e4",
  "filename": "Research_Paper.pdf",
  "word_count": 4250,
  "chunk_count": 11,
  "status": "indexed_in_qdrant"
}
```

3 Authentication & Session Endpoints

3.1 User Registration & Login

Endpoint	Method	Payload / Params	Response
`/api/auth/register`	`POST`	`{"username": "...", "password": "..."}`	User profile created (201 Created)
`/api/auth/token`	`POST`	`{"username": "...", "password": "..."}`	`{"access_token": "...", "token_type": "bearer"}`
`/api/health`	`GET`	None	`{"status": "healthy", "version": "2.0"}`

4 Structured Output Schemas (Code & Resume Reviewer)

For structured personas (`code_reviewer` and `resume_reviewer`), the API guarantees strict validated JSON output:

```
// Code Reviewer Validated Schema Output:
{
  "language": "python",
  "issues": [
    {
      "category": "security",
      "severity": "high",
      "line": 2,
      "title": "SQL Injection Risk",
      "description": "Direct string concatenation in SQL query.",
      "fix": "Use parameterized queries with db.execute(query, (user_id,))"
    }
  ],
  "summary": "1 high severity security issue found. Logic and style are clean."
}
```

5 HTTP Error Codes & Handlers

HTTP Code	Error Type	Resolution & Behavior
`200 OK`	Success / SSE Stream	Normal streaming connection established.
`400 Bad Request`	Invalid Payload Schema	Missing prompt or malformed history array.
`401 Unauthorized`	Invalid / Expired JWT	Token missing or signature invalid.
`413 Payload Too Large`	File Size Exceeded	Uploaded PDF exceeds max limit (10MB).
`429 Rate Limited`	Token Bucket Exhausted	Automatic upstream hot-swap to Groq fallback.
`500 Server Error`	Internal Exception	Fallback error response returned safely.